

your handwritten code. In such situations you won't be archiving or throwing away the original source. Here it makes sense to put things in a pipeline. The *Makefile* in our C enum stringification example is an example for this. The source file is checked for updates every time we build the software and the generation commands are run if there is something new.

But what if you had to make manual changes to the generated code and still want it auto-updated whenever the source changes? Let's see an example.

Don't give up on the pipeline: kompose example

kompose is a tool that helps you generate Kubernetes YAML files from Docker Compose files. It is intended to be a one-time process because there is no one-to-one mapping between Docker Compose and Kubernetes. So you are guaranteed to be making changes to the generated files. But what if you want to keep both Docker Compose and Kubernetes versions in parallel, auto-applying changes in the Docker Compose while still keeping your edits to the generated configuration?

I had faced a similar challenge. These were my additions:

1. k8s image pull policy and image pull secret, which were missing in the Docker Compose version
2. Services ignored by kompose because the ports are not exposed in the Docker Compose version
3. Configuration for *initContainers*, which was missing in the Docker Compose version

Instead of choosing to convert once and keep both versions in sync manually, I chose the following solutions, respectively, for each of the above:

1. Use kompose annotations *kompose.image-pull-policy*, *kompose.image-pull-secret* in the Docker Compose file itself, so that kompose knows to add them.
2. Write files for missing services manually.
3. Use a patch file to add the *initContainers* section.

This way, things will be reproducible while still keeping customisations. Of course it doesn't make sense to do this if the differences outnumber similarities.

nguigen

I've been developing desktop applications for more than a decade. Although trivial, they've been used by others, which means I had to regularly update, port and package these apps. *nguigen* (short: *ngg*) is a project that I started as a solution to several difficulties I experienced as part of this.

It is a general-purpose programming language and a code generator that aims to generate code for multiple languages, platforms and GUI toolkits from a single, mostly generic codebase. This is different from solutions like Electron because we're going native here and there is zero

overhead. Once mature enough, it could be used to generate C/GTK, C++/Qt, Android and Web apps.

Its C output is already mature and is being used to self-host the compiler (i.e., *ngg* compiler is written in *ngg* itself). *nguigen* offers more expressiveness and compile-time semi-automatic memory management when generating C code. This is important since the C ecosystem remains far superior to many alternatives and one would choose to stick with it.

nguigen is not released yet, but the plan is to release it as free (libre) software. Learn more at nandakumar.co.in/software/nguigen/.

Codegen in the nguigen ecosystem

The development of *nguigen* itself depends on code generation. Apart from being written in itself and self-compiled to C, parts of the source are auto-generated from mappings written in TSV (tab-separated values) files.

Makefiles with correct dependencies are auto generated using *ngg* itself and a bash script. This comes handy during major refactorings.

The parser is also auto-generated from grammar specs. Usually lex and bison are used for this purpose, but for *nguigen*, I wrote my own.

And, finally, there is *h2ngg*, a simple tool to auto-generate *ngg* interfaces (like header files) for external libraries like GTK.

Write your own code generators

Maybe you've already noticed it: this talk is not just about existing code generators. It's about the advantages and fun of generative programming, which can only be enjoyed at its maximum if you write your own generators. They benefit from being problem-specific.

How to do that? Maybe you can begin with Makefiles and shell scripts. Use some Python. lex and bison will be overkill, unless you are trying to create a language. **END** 

References

- Slides of the talk: nandakumar.org/talks/
- Demo repo: github.com/nandedamana/lazy-becomes-prolific
These resources cover several things not covered in this article. The demo repo contains working examples that you can try yourself. It is supposed to be updated also.

By: Nandakumar Edamana

The author is a developer and technology writer who primarily focuses on software freedom, privacy and other ethical issues. He has authored a couple of books and several articles for mainstream publishers. A GNU/Linux user for 15+ years, he releases his software products under free software licences.